Uppsala University



Machine Learning

Assignment 4

Contents

1 Oral exam question contribution 5

2 Convolutional neural networks using Keras 6

3 * Implementing a convolutional layer 10

4 * Transfer learning using VGG16 13

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution. State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available as **a PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

  1. `install.packages("remotes")`
  2. ```
     remotes::install_github("MansMeg/IntroML",
     subdir = "rpackage")
     ```
     ```

- Many of the exercises can be checked automatically using the R package `markmyassignment`. Information on how to install and use the package can be found **here**. *Don't include* ***`markmyassignment`*** *results in the report.*
- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- You are not allowed to show your assignments (text or code) to anyone (including LLMs). Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Automatic feedback with `markmyassignment`

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment4/assignment4-vg.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
```

```
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected tasks. They do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. *You should not need to include MMA results in the report.* The supplied checks run R code. Python implementations are assessed from the report and cannot be run directly through MMA.

These checks cover the three functions in the manual convolution VG option. The Keras model-building and transfer-learning tasks are assessed from your report.

Keras, TensorFlow, and Google Colab

This assignment uses `keras3` and `tensorflow` for R. Use the `keras3` package for all Keras code. Adam is the default optimizer: use `optimizer_adam(learning_rate = 0.001)` unless the task or supplied example specifies another learning rate or explicitly compares optimizers.

To work in Google Colab, open the course R setup notebook, save a copy in Google Drive, and run its setup and training-check cells. Wait for "PASS: training and prediction work" before starting the assignment.

Use the assignment report template linked in the notebook to structure your report. Write explanations in text cells and put R code in code cells. Save your notebook and results in Drive, and run the setup cell again at the start of each R session.

For local installation, follow the course R setup guide. Details on the architecture components are available in the `keras3` reference.

---

1 Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 1–3 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

2 Convolutional neural networks using Keras

Background reading: Follow the general reading instructions for Block 4.

The random-search guidance is adapted from Goodfellow et al. (2016, Table 11.1 and Section 11.4.4). The procedure is specified below; no reading of that book is required.

We are now going to implement a convolutional neural network using Keras. Here Ch. 5.1-5.3 in Chollet and Allaire (2018) and the Keras 3 CIFAR-10 example might be useful for getting started. Remember to load the `tensorflow` R package before loading the `keras3` R package.

If you have installed `keras3`, you can load the MNIST dataset. This dataset contains data on handwritten digits that we want to classify. To access the data, we use `keras3` as follows.

```
library(tensorflow)
library(keras3)
mnist <- dataset_mnist()
mnist$train$x <- mnist$train$x/255
mnist$test$x <- mnist$test$x/255
```

1. (1p) Display training observations 1, 2, and 3, with their true digit labels, and include these images in your report.

```
idx <- 1
im <- mnist$train$x[idx,,]
# Transpose the image
im <- t(apply(im, 2, rev))
image(1:28, 1:28, im, col=gray((0:255)/255), xlab = "", ylab = "",
        xaxt='n', yaxt='n', main=paste(mnist$train$y[idx]))
```

2. (1p) Implement a Convolutional Neural Network for the MNIST dataset. The network should have two convolutional layers as follows.

```
## _________________________________________________________________
## Layer (type)                 Output Shape              Param #
## =================================================================
## conv2d (Conv2D)              (None, 26, 26, 32)        320
## _________________________________________________________________
## max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
## _________________________________________________________________
## conv2d (Conv2D)              (None, 11, 11, 32)        9248
## _________________________________________________________________
## flatten (Flatten)            (None, 3872)              0
## _________________________________________________________________
## dense (Dense)                (None, 64)                247872
## _________________________________________________________________
## dense (Dense)                (None, 10)                650
## =================================================================
## Total params: 258,090
## Trainable params: 258,090
## Non-trainable params: 0
```

Example: Counting convolutional parameters *No submission required*

Four 2×2 filters applied to three input channels have

$$4(2 \cdot 2 \cdot 3 + 1) = 52$$

parameters. Each filter spans all input channels, giving $2 \cdot 2 \cdot 3$ kernel weights and one bias per filter.

3. (1p) Explain why there are 320 parameters in the first layer. How many are kernel weights (and why), and how many biases?

4. (1p) Train the network and use `evaluate()` on the MNIST test set to report its **test loss and test accuracy**. Include your training code once.

5. (3p) Choose and briefly justify at least three hyper-parameters and their sampling distributions. Use random search to train 10 fresh models: for each trial, draw a setting from each chosen distribution and train a newly initialized model using those settings. Keep the training/validation split and number of epochs fixed, and record each model's settings and final-epoch validation accuracy. Select the best setting using validation accuracy, then report the code and test accuracy of your final model. Keep the test data out of the search.

   Examples of hyper-parameters you may include are:

   - **Learning rate:** sample uniformly on a logarithmic scale between 10^{-4} and 10^{-2}.
   - **Filters per convolutional layer:** sample uniformly from $\{16, 32, 64\}$.
   - **Batch size:** sample uniformly from $\{32, 64, 128\}$.
   - **Dense hidden units:** sample uniformly from $\{32, 64, 128\}$.
   - **Dropout rate:** sample uniformly between 0 and 0.5, if you include a dropout layer.

These are suggestions; other justified choices are also allowed. Adapt the scaffold below to the hyper-parameters you choose.

Example: A random-search loop *No submission required*

This scaffold uses `keras3`. First prepare fixed training and validation arrays `x_train` and `x_val` with dimensions `c(n, 28, 28, 1)`, scaled to $[0, 1]$, and matching label vectors with values 0–9. Write `build_model(filters, learning_rate)` to create and compile a fresh CNN using sparse categorical cross-entropy and the supplied settings. The choices below are illustrative; adapt them to your search.

```
set.seed(4711)
settings <- data.frame(
  filters = sample(c(16L, 32L, 64L), 10, replace = TRUE),
  learning_rate = 10^runif(10, -4, -2),
  batch_size = sample(c(32L, 64L, 128L), 10, replace = TRUE)
)
settings$val_accuracy <- NA_real_
for (i in seq_len(nrow(settings))) {
  clear_session()
  model <- build_model(settings$filters[i],
                       settings$learning_rate[i])
  history <- fit(model, x_train, y_train,
    validation_data = list(x_val, y_val),
    epochs = 5, batch_size = settings$batch_size[i],
    verbose = 0)
  settings$val_accuracy[i] <-
    tail(history$metrics$val_accuracy, 1)
}
best <- settings[which.max(settings$val_accuracy), ]
best
```

Rebuild the selected configuration and fit it before evaluating it on the test data. Here `10^runif(10, -4, -2)` samples learning rates uniformly on a logarithmic scale, from 10^{-4} to 10^{-2}.

6. (1p) Load CIFAR-10 using `dataset_cifar10()` (see the Keras 3 CIFAR-10 example). Visualize one image of your choice together with its class/category.

   Hint! Display the first unscaled training image as RGB as follows. Change the index 1 to choose another image; use `max = 1` if you have already scaled the pixels to $[0, 1]$.

```
cifar10 <- dataset_cifar10()
plot(as.raster(cifar10$train$x[1, , , ], max = 255))
```

7. (1p) Implement the following CNN as your *baseline* for CIFAR-10; reuse it in the comparison below.

```
## Model: "sequential"
## ________________________________________________________________________
## Layer (type)                        Output Shape                    Param #
## ========================================================================
## conv2d (Conv2D)                     (None, 30, 30, 32)              896
## ________________________________________________________________________
## max_pooling2d (MaxPooling2D)        (None, 15, 15, 32)              0
## ________________________________________________________________________
## conv2d_1 (Conv2D)                   (None, 13, 13, 64)              18496
## ________________________________________________________________________
## max_pooling2d_1 (MaxPooling2D)      (None, 6, 6, 64)                0
## ________________________________________________________________________
## conv2d_2 (Conv2D)                   (None, 4, 4, 64)                36928
## ________________________________________________________________________
## flatten (Flatten)                   (None, 1024)                    0
## ________________________________________________________________________
## dense (Dense)                       (None, 64)                      65600
## ________________________________________________________________________
## dense_1 (Dense)                     (None, 10)                      650
## ========================================================================
## Total params: 122,570
## Trainable params: 122,570
## Non-trainable params: 0
## ________________________________________________________________________
```

   Note! This problem is more complex than the MNIST problem, so don't be surprised if you get lower accuracy than you got on the MNIST dataset.

8. (1p) Why do we now have 896 parameters in the first convolutional layer?

9. (2p) Train and compare three CIFAR networks: the baseline above and two variations, each making one motivated change to the baseline (for example, changing the number of filters or adding dropout). Keep the training/validation split, preprocessing, optimizer, learning rate, batch size, and number of epochs fixed. Reuse the baseline run. Include each model's Keras summary and a table of training and validation accuracies at the final epoch. Briefly state your expected effect and explain the observed difference from the baseline; an improvement is not required. Use validation results for the comparison and keep the test data out of model selection.

3 * Implementing a convolutional layer

For VG, complete either this section or Section 4. You may do both, but only one VG point is awarded. Report the time spent on one chosen option.

As a first step, we will implement a layer of a convolutional neural network with one filter. We will here not train the layer, only implement it to understand the inner workings. You are not allowed to use `convolve()` in R in the final solution, but you can use it to debug your code if you want to.

The convolution explanation is adapted from Goodfellow et al. (2016, Figure 9.1). All implementation details needed here are given below; no reading of that book is required. The video Ng (2017), listed in the Block 4 reading instructions, provides a visual explanation.

For an $R \times C$ image X and an $m \times m$ kernel K, compute each filter response as

$$S_{ij} = \sum_{u=1}^{m} \sum_{v=1}^{m} X_{i+u-1,j+v-1} K_{uv}.$$

Slide the kernel one row or column at a time, without padding or flipping the kernel. The result has dimensions $(R-m+1) \times (C-m+1)$. This is the CNN convention, also called cross-correlation. A convolutional layer then applies activation$(S_{ij}+b)$ separately at every position. For max-pooling, take the maximum in each non-overlapping 2×2 window with stride 2, discarding any incomplete border windows.

Start by importing examples from the MNIST dataset as follows.

```
library(uuml)
data("mnist_example")
```

To visualize an image use:

```
im <- mnist_example[["4"]]
image(1:ncol(im), 1:nrow(im), im,
      xlab = "", ylab = "",
      xaxt='n', yaxt='n', main="4")
```

Plotting orientation: In `image()`, matrix rows correspond to the x-axis and columns to the y-axis. Interpret filter directions in the plotted image's coordinates, and use the same plotting convention for the original image and its feature maps.

1. (1p) Visualize the MNIST example digit 4.
2. (3p) **Implementation (2p).** Implement `convolution(X, K)` to take an MNIST image `X` and a square kernel `K` and return a feature map. Use stride 1 and no padding ("valid" convolution). **Example (1p).** Include the inputs, a call and actual output from your implementation, with a brief explanation of what it checks. Reuse the supplied example below or a later required feature-map calculation.

```r
X <- mnist_example[["4"]][12:15,12:15]
X
```

```
##      [,1] [,2] [,3] [,4]
## [1,]   56  250  116    0
## [2,]    0  240  144    0
## [3,]    0  198  150    0
## [4,]    0  143  241    0
```

```r
K <- matrix(c(-0.5, -0.5, -0.5,
              1,  1,  1,
              -0.5, -0.5, -0.5), nrow = 3, byrow = TRUE)
K
```

```
##      [,1] [,2] [,3]
## [1,] -0.5 -0.5 -0.5
## [2,]  1.0  1.0  1.0
## [3,] -0.5 -0.5 -0.5
```

```r
convolution(X, K)
```

```
##      [,1] [,2]
## [1,]   -1   27
## [2,]  -36  -36
```

3. (1p) Visualize the feature map of MNIST example digit 4 using the above three by three kernel `K`.

4. (3p) **Implementation (2p).** Implement the convolutional layer.

   The function `convolutional_layer(X, K, b, activation)` should combine convolution, bias addition and the supplied activation function.

   Example (1p). Include the inputs, a call and actual output from your implementation, with a brief explanation of what it checks. Reuse the supplied example below or a later required feature-map calculation.

```r
relu <- function(x) max(0, x)
convolutional_layer(X, K, 100, relu)
```

```
##      [,1] [,2]
## [1,]   99  127
## [2,]   64   64
```

Example: Bias as a threshold before ReLU *No submission required*

For a filter response of 200 before adding the bias,

$$\mathrm{ReLU}(200 - 150) = 50, \qquad \mathrm{ReLU}(200 - 370) = 0.$$

A more negative bias suppresses weaker responses. The fixed biases below illustrate this thresholding; a trained CNN learns its biases. Keep the supplied `mnist_example` pixel scale of 0–255 here: rescaling pixels also changes the filter responses.

5. (2p) Run your convolutional layer on MNIST example digit 4 with bias -150. Visualize the feature map as you visualize the original image. What does the filter seem to capture?

6. (1p) Now transpose your filter and run your convolutional layer on MNIST example digit 4 with bias -150. Visualize the feature map. What does that transposed filter seem to capture?

7. (3p) **Implementation (2p).** Implement `maxpool_layer(X)` as a two by two max-pooling layer with stride 2.

   Example (1p). Include the inputs, a call and actual output from your implementation, with a brief explanation of what it checks. Reuse the supplied example below or the final pooled feature-map calculation.

```
X <- mnist_example[["4"]][12:15,12:15]
maxpool_layer(X)
```

```
##      [,1] [,2]
## [1,]  250  144
## [2,]  198  241
```

8. (1p) Put it all together and visualize the final pooled feature map.

```
X <- mnist_example[["4"]]
output <- maxpool_layer(convolutional_layer(X, K, -370, relu))
```

4 * Transfer learning using VGG16

This is the alternative VG option; see Section 3 for the choice and time-reporting instructions.

As the last step, we will look into using transfer learning as a quick way of improving the prediction accuracy on the cifar-10 dataset. Here Ch. 5.3 in Chollet and Allaire (2018) or the `keras3` transfer-learning guide might be helpful.

1. (2p) Using `application_vgg16()`, load the ImageNet-pretrained convolutional base with `include_top = FALSE` and `input_shape = c(32, 32, 3)`. Store it as `conv_base`.

   Hint! With raw CIFAR images stored in `cifar10`, use VGG16 preprocessing as below. It converts RGB to BGR and subtracts the ImageNet channel means from the raw 0–255 values. This replaces the division by 255 used earlier. Apply the same preprocessing to any validation images.

   ```
   freeze_weights(conv_base) # Before compiling the full model
   x_train_vgg <- application_preprocess_inputs(
     conv_base, cifar10$train$x)
   x_test_vgg <- application_preprocess_inputs(
     conv_base, cifar10$test$x)
   ```

2. (2p) Add a flattening layer, a dense layer with 64 hidden units and ReLU activation, and a 10-class softmax output. Include the full model summary. Compare the number of trainable parameters in this dense top with the dense top of the CIFAR-10 baseline.

3. (2p) Reuse the CIFAR-10 training/validation split. Compile the full model after freezing the convolutional base, then train only the dense top on the preprocessed images for five epochs. Runtime depends on your hardware; roughly 200 seconds per epoch or more is possible. Time the first of these five epochs to estimate the remaining four. The first epoch may include additional setup time.

4. (2p) Report the VGG16 model's **test accuracy** and compare it with the **CIFAR-10 baseline** on the same held-out test images, using each model's required preprocessing.

References

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

François Chollet and Joseph J Allaire. *Deep Learning with R*. Manning, 2018.

Andrew Ng. C4w1l07 one layer of a convolutional net, 2017. URL https://www.youtube.com/watch?v=jPOAS7uCODQ&list=PLkDaE6sCZn6Gl29AoE31iwdVwSG-KnDzF&index=8.